Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №10

“ Production Python: Docker, CI/CD та захист проєкту”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета роботи : Ознайомитися з підходами до підготовки Python-проєкту до production-oriented використання.

Варіант 10. Система обліку складських запасів

• Продовжити той самий проєкт Lab_01. Основні дані: код, назва, кількість, ціна.

• Зберегти предметну функціональність: надходження; списання; пошук; загальна вартість; низькі залишки; історія руху; заборона надмірного списання.

• Production package/module: warehouse-manager / warehouse_manager.

• REST API, створене у Lab_08, зберегти та задокументувати в README: /products; /products/{id}/incoming; /products/{id}/write-off; /stock/operations; обов'язково додати GET /health.

• Використати persistence layer і БД з Lab_07 та оптимізації, що залишені після експериментів Lab_09. Конфігурацію БД і застосунку передавати через environment variables.

• Мінімальні environment variables: DATABASE_URL, APP_ENV або ENVIRONMENT, LOG_LEVEL. Реальні passwords, tokens, API keys та credentials не зберігати у source code, README, Dockerfile або Git.

• Створити pyproject.toml metadata, wheel і sdist; перевірити встановлення wheel у чистому virtual environment.

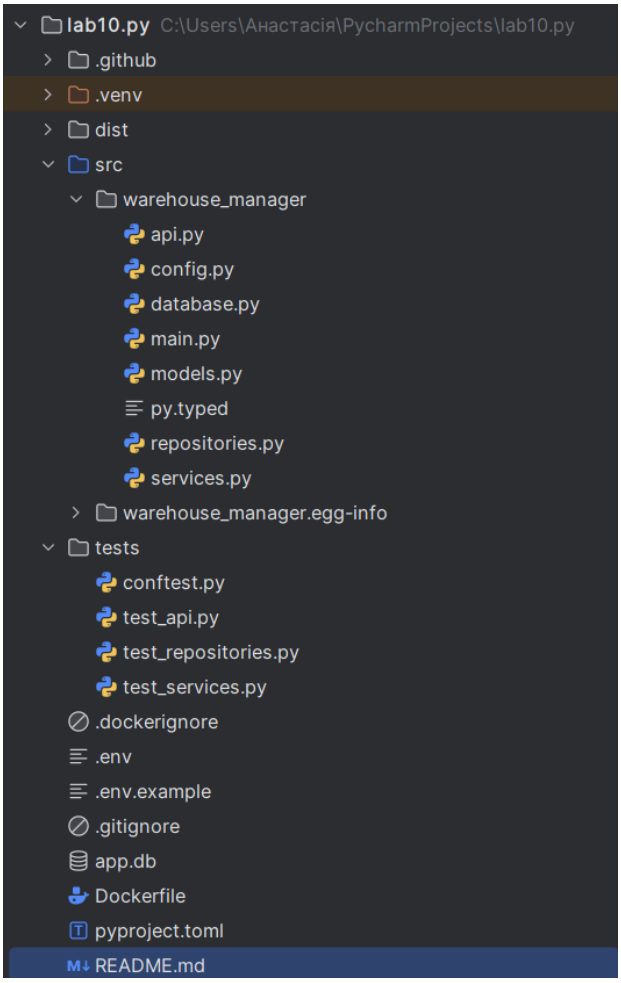
• Створити Dockerfile і .dockerignore, побудувати image, запустити container; для SQLite або іншої локальної БД забезпечити persistent volume, якщо це потрібно проєкту.

• GitHub Actions повинен автоматично виконувати ruff check, format check, mypy, pytest, package build і docker build.

• README повинен описувати предметну систему, installation, configuration, local/Docker run, API, tests, CI/CD та сценарій демонстрації саме цього варіанта.

Хід роботи:

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src/warehouse_manager/ зосередила весь програмний код (базові моделі, репозиторії, сервіси, конфігурацію та API на FastAPI), залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, .env.example, .gitignore, README.md, Dockerfile), каталогу тестів tests/



2. **pyproject.toml** — Я налаштувала метадані пакета, вказала версію проєкту, інформацію про автора, мінімальну версію Python, а також основні виробничі залежності (FastAPI, Uvicorn, Pydantic, SQLAlchemy, pydantic-settings). Також додала розробницькі залежності (pytest, pytest-asyncio, mypy, ruff) та вказала конфігурацію збірки проєкту за допомогою setuptools через src-layout.

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "warehouse-manager"
version = "1.0.0"
description = "Warehouse management REST API"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "fastapi>=0.110",
    "uvicorn>=0.30",
    "sqlalchemy>=2.0",
    "pydantic>=2.0",
    "pydantic-settings>=2.0",
    "httpx>=0.27",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "pytest-asyncio>=0.23",
    "ruff>=0.5",
```

```
    "mypy>=1.10",
    "build>=1.2",
]

[project.scripts]
warehouse-manager = "warehouse_manager.main:main"

[tool.ruff]
line-length = 88

[tool.mypy]
python_version = "3.11"
strict = true
ignore_missing_imports = true
[[tool.mypy.overrides]]
module = "warehouse_manager.*"
ignore_missing_imports = true
```

3. **README.md** — Я описала призначення проєкту («Система обліку складських запасів» / warehouse-manager), інструкції зі встановлення, налаштування та приклади запуску для користувачів і розробників.

```
# Warehouse Manager (Система обліку складських запасів)

REST API застосунок для автоматизації та обліку складських операцій, побудований на базі **FastAPI**, **SQLAlchemy** та **Pydantic**. Проєкт повністю підготовлений до розгортання в production-середовищі (включно з контейнеризацією через Docker, управлінням через змінні середовища та CI/CD на GitHub Actions).

---

## 📋 Основні можливості (Features)
* **Управління товарами:** Створення нових товарів, перегляд списку запасів.
* **Складські операції:** Облік надходження товарів на склад (`incoming`) та списання (`write-off`).
* **Захист від помилок:** Валідація кількості запасів (заборона надмірного списання при нестачі товару).
* **Персистентність:** Зберігання даних за допомогою SQLite / реляційної бази даних.
* **Health Check:** Ендпоінт для моніторингу працездатності сервісу (`/health`).
* **Автоматизована перевірка та збірка:** Повноцінний CI/CD pipeline через GitHub Actions.

---

## 🛠 Технологічний стек
* **Python** 3.11+
* **FastAPI** (сучасний асинхронний вебфреймворк)
* **SQLAlchemy** (ORM для роботи з БД)
* **Pydantic / Pydantic Settings** (валідація даних та конфігурація)
* **Docker** (контейнеризація)
* **Ruff & Mypy** (лінтування та статична перевірка типів)
* **Pytest** (модульне тестування)

---

## ⚙ Вимоги та встановлення (Installation)

1. Клонуйте репозиторій та перейдіть до папки проєкту:
   ```bash
   git clone [https://github.com/nainastiia/warehouse-
manager.git](https://github.com/nainastiia/warehouse-manager.git)
   cd warehouse-manager
   ```
```

```
2. Створіть та активуйте віртуальне середовище:
   ```bash
   python -m venv .venv
   source .venv/bin/activate  # На Windows: .venv\Scripts\activate
   ```
3. Встановіть пакет у режимі розробки разом із інструментами для тестування
(dev-dependencies):
   ```bash
   pip install --upgrade pip
   pip install -e ".[dev]"
   ```
4. Налаштування змінних середовища (.env)
cp .env.example .env

5. Запуск юніт-тестів та перевірка якості коду (лінт + типізація)
pytest -v
ruff check
ruff format --check
mypy src

6. Збірка дистрибутивних пакетів (wheel та sdist)
python -m build

7. Локальний запуск REST API застосунку
warehouse-manager
Або через uvicorn напряму:
uvicorn warehouse_manager.api:app --reload

8. Збірка та запуск у Docker-контейнері (в іншій вкладці термінала)
docker build -t warehouse-manager:1.0 .
docker run --rm -p 8000:8000 \
  -v warehouse_data:/app/data \
  -e ENVIRONMENT=production \
  -e LOG_LEVEL=INFO \
  -e DATABASE_URL=sqlite:////app/data/app.db \
  warehouse-manager:1.0
```

4. **src/warehouse_manager/models.py** — Я реалізувала SQLAlchemy-моделі даних для обліку товарів та складських операцій, успадкувавши їх від базового класу (DeclarativeBase), а також налаштувала поля таблиць і типи даних.

```
import datetime

from sqlalchemy import Column, DateTime, Float, ForeignKey, Integer, String
from sqlalchemy.orm import DeclarativeBase, relationship


class Base(DeclarativeBase):
    pass


class ProductModel(Base):
    __tablename__ = "products"

    id = Column(Integer, primary_key=True, index=True)
    code = Column(String, unique=True, index=True, nullable=False)
    name = Column(String, nullable=False)
    quantity = Column(Integer, default=0, nullable=False)
    price = Column(Float, nullable=False)

    operations = relationship(
        "StockOperationModel", back_populates="product", cascade="all, delete-
orphan"
```

```
    )


class StockOperationModel(Base):
    __tablename__ = "stock_operations"

    id = Column(Integer, primary_key=True, index=True)
    product_id = Column(Integer, ForeignKey("products.id"), nullable=False)
    operation_type = Column(String, nullable=False)  # "incoming" або "write-
off"
    amount = Column(Integer, nullable=False)
    timestamp = Column(DateTime, default=datetime.datetime.utcnow,
nullable=False)

    product = relationship("ProductModel", back_populates="operations")
```

5. **src/warehouse_manager/database.py** — Я налаштувала підключення до бази даних (SQLite), створила рушій (engine) та налаштувала фабрику сесій (SessionLocal) для взаємодії з репозиторіями і сервісами.

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from warehouse_manager.config import get_settings

settings = get_settings()

engine = create_engine(
    settings.database_url,
    connect_args={"check_same_thread": False}
    if "sqlite" in settings.database_url
    else {},
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    expire_on_commit=False,
)
```

6. **src/warehouse_manager/config.py** — Я реалізувала управління конфігурацією та змінними середовища за допомогою pydantic-settings, визначивши налаштування застосунку, середовища, рівня логування та рядка підключення до БД.

```
from functools import lru_cache

from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    app_name: str = "Warehouse Manager"
    environment: str = "development"
    database_url: str = "sqlite:///app.db"
    log_level: str = "INFO"

    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-
8")


@lru_cache
```

```
def get_settings() -> Settings:
    return Settings()
```

7. **src/warehouse_manager/repositories.py** — Я реалізувала шаблон репозиторія для інкапсуляції логіки роботи з базою даних, включаючи створення товарів, отримання інформації про них та реєстрацію складських рухів (надходжень і списань).

```
from sqlalchemy.orm import Session

from warehouse_manager.models import ProductModel, StockOperationModel


class WarehouseRepository:
    def __init__(self, db: Session):
        self.db = db

    def get_all_products(self) -> list[ProductModel]:
        return self.db.query(ProductModel).all()

    def get_product_by_id(self, product_id: int) -> ProductModel | None:
        return self.db.query(ProductModel).filter(ProductModel.id ==
product_id).first()

    def create_product(
        self, code: str, name: str, quantity: int, price: float
    ) -> ProductModel:
        product = ProductModel(code=code, name=name, quantity=quantity,
price=price)
        self.db.add(product)
        self.db.commit()
        self.db.refresh(product)
        return product

    def add_incoming(self, product_id: int, amount: int) -> ProductModel:
        product = self.get_product_by_id(product_id)
        if product:
            product.quantity += amount
            operation = StockOperationModel(
                product_id=product_id, operation_type="incoming", amount=amount
            )
            self.db.add(operation)
            self.db.commit()
            self.db.refresh(product)
        return product

    def write_off(self, product_id: int, amount: int) -> ProductModel | None:
        product = self.get_product_by_id(product_id)
        if product and product.quantity >= amount:
            product.quantity -= amount
            operation = StockOperationModel(
                product_id=product_id, operation_type="write-off", amount=amount
            )
            self.db.add(operation)
            self.db.commit()
            self.db.refresh(product)
            return product
        return None

    def get_operations(self) -> list[StockOperationModel]:
        return self.db.query(StockOperationModel).all()
```

8. **src/warehouse_manager/services.py** — Я реалізувала бізнес-логіку застосунку, додавши перевірки валідності кількості товарів під час операцій надходження та списання, а також обробку виняткових ситуацій.

```
from sqlalchemy.orm import Session

from warehouse_manager.models import ProductModel, StockOperationModel
from warehouse_manager.repositories import WarehouseRepository


class WarehouseService:
    def __init__(self, db: Session):
        self.repo = WarehouseRepository(db)

    def list_products(self) -> list[ProductModel]:
        return self.repo.get_all_products()

    def register_incoming(self, product_id: int, amount: int) -> ProductModel:
        if amount <= 0:
            raise ValueError("Кількість для надходження має бути більше нуля.")
        product = self.repo.add_incoming(product_id, amount)
        if not product:
            raise ValueError("Товар не знайдено.")
        return product

    def register_write_off(self, product_id: int, amount: int) -> ProductModel:
        if amount <= 0:
            raise ValueError("Кількість для списання має бути більше нуля.")
        product = self.repo.write_off(product_id, amount)
        if not product:
            raise ValueError(
                "Товар не знайдено або недостатньо коштів/запасів на складі
(надмірне списання)."
            )
        return product

    def list_operations(self) -> list[StockOperationModel]:
        return self.repo.get_operations()
```

9. **src/warehouse_manager/api.py** — Я реалізувала веб-інтерфейс на базі FastAPI, створивши маршрути (endpoints) для перевірки стану (/health), роботи з товарами (/products), проведення складських операцій та отримання історії рухів запасів.

```
import logging
from typing import Any

from fastapi import Depends, FastAPI, HTTPException, status
from pydantic import BaseModel
from sqlalchemy.orm import Session

from warehouse_manager.config import get_settings
from warehouse_manager.database import SessionLocal, engine
from warehouse_manager.models import Base
from warehouse_manager.repositories import WarehouseRepository
from warehouse_manager.services import WarehouseService

settings = get_settings()

# Створення таблиць БД при старті
Base.metadata.create_all(bind=engine)
```

```
logging.basicConfig(
    level=settings.log_level,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
)
logger = logging.getLogger("warehouse_manager")
logger.info("Warehouse Manager started")

app = FastAPI(title=settings.app_name)


def get_db() -> Any:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()


class ProductCreate(BaseModel):
    code: str
    name: str
    quantity: int
    price: float


class OperationRequest(BaseModel):
    amount: int


@app.get("/health")
async def health() -> dict[str, str]:
    return {
        "status": "ok",
        "environment": settings.environment,
    }


@app.get("/products")
async def get_products(db: Session = Depends(get_db)) -> Any:
    service = WarehouseService(db)
    return service.list_products()


@app.post("/products", status_code=status.HTTP_201_CREATED)
async def create_product(data: ProductCreate, db: Session = Depends(get_db)) ->
Any:
    repo = WarehouseRepository(db)
    product = repo.create_product(data.code, data.name, data.quantity,
data.price)
    return product


@app.post("/products/{id}/incoming")
async def product_incoming(
    id: int, data: OperationRequest, db: Session = Depends(get_db)
) -> Any:
    service = WarehouseService(db)
    try:
        return service.register_incoming(id, data.amount)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))


@app.post("/products/{id}/write-off")
async def product_write_off(
```

```
    id: int, data: OperationRequest, db: Session = Depends(get_db)
) -> Any:
    service = WarehouseService(db)
    try:
        return service.register_write_off(id, data.amount)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))


@app.get("/stock/operations")
async def get_stock_operations(db: Session = Depends(get_db)) -> Any:
    service = WarehouseService(db)
    return service.list_operations()
```

10. **tests/conftest.py** — Я налаштувала загальні фікстури для pytest, включаючи створення тестової бази даних в пам'яті (sqlite:///:memory:), налаштування тестової сесії та клієнта FastAPI (TestClient) для інтеграційних тестувань.

```
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from warehouse_manager.api import app, get_db
from warehouse_manager.models import Base

SQLALCHEMY_DATABASE_URL = "sqlite:///:memory:"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)


@pytest.fixture(autouse=True)
def setup_database():
    Base.metadata.create_all(bind=engine)
    yield
    Base.metadata.drop_all(bind=engine)


@pytest.fixture
def db_session(setup_database):
    connection = engine.connect()
    transaction = connection.begin()
    session = TestingSessionLocal(bind=connection)
    yield session
    session.close()
    transaction.rollback()
    connection.close()


@pytest.fixture(autouse=True)
def override_dependency(db_session):
    def override_get_db():
        try:
            yield db_session
        finally:
            pass

    app.dependency_overrides[get_db] = override_get_db
```

```
    yield
    app.dependency_overrides.clear()


@pytest.fixture
def client():
    return TestClient(app)
```

11. **tests/test_api.py**, **tests/test_repositories.py**, **tests/test_services.py** — Я реалізувала модульні та інтеграційні тести для перевірки коректності роботи API, репозиторіїв та бізнес-логіки сервісів.

test_api.py

```
def test_health(client):
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"
```

test_repositories.py

```
from sqlalchemy.orm import Session

from warehouse_manager.repositories import WarehouseRepository


def test_create_and_get_product(db_session: Session):
    repo = WarehouseRepository(db_session)
    product = repo.create_product(
        code="PRD-001", name="Ноутбук", quantity=10, price=25000.0
    )

    assert product.id is not None
    assert product.code == "PRD-001"
    assert product.name == "Ноутбук"
    assert product.quantity == 10

    fetched = repo.get_product_by_id(product.id)
    assert fetched is not None
    assert fetched.code == "PRD-001"


def test_add_incoming_and_write_off(db_session: Session):
    repo = WarehouseRepository(db_session)
    product = repo.create_product(code="PRD-002", name="Мишка", quantity=5,
price=500.0)

    # Надходження товару
    updated = repo.add_incoming(product.id, 10)
    assert updated.quantity == 15

    # Списання товару
    written_off = repo.write_off(product.id, 4)
    assert written_off is not None
    assert written_off.quantity == 11

    # Списання занадто великої кількості (надмірне списання)
    failed_write_off = repo.write_off(product.id, 20)
    assert failed_write_off is None
```

test_services.py

```
import pytest
from sqlalchemy.orm import Session

from warehouse_manager.repositories import WarehouseRepository
```

```python
from warehouse_manager.services import WarehouseService


def test_service_register_incoming(db_session: Session):
    repo = WarehouseRepository(db_session)
    product = repo.create_product(
        code="PRD-003", name="Монітор", quantity=2, price=7000.0
    )

    service = WarehouseService(db_session)
    updated = service.register_incoming(product.id, 3)
    assert updated.quantity == 5


def test_service_invalid_incoming_amount(db_session: Session):
    repo = WarehouseRepository(db_session)
    product = repo.create_product(
        code="PRD-004", name="Клавіатура", quantity=5, price=1500.0
    )

    service = WarehouseService(db_session)
    with pytest.raises(
        ValueError, match="Кількість для надходження має бути більше нуля"
    ):
        service.register_incoming(product.id, 0)


def test_service_excessive_write_off(db_session: Session):
    repo = WarehouseRepository(db_session)
    product = repo.create_product(
        code="PRD-005", name="Навушники", quantity=2, price=2000.0
    )

    service = WarehouseService(db_session)
    with pytest.raises(
        ValueError, match="Товар не знайдено або недостатньо коштів/запасів"
    ):
        service.register_write_off(product.id, 10)
```

12. **Dockerfile** — Я створила інструкції для контейнеризації застосунку, налаштувавши середовище виконання Python, встановлення залежностей та запуск вебсервера.

```dockerfile
FROM python:3.12-slim AS builder
WORKDIR /build
COPY pyproject.toml README.md ./
COPY src/ ./src
RUN pip install --no-cache-dir build && python -m build

FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY --from=builder /build/dist/*.whl /tmp/
RUN pip install --no-cache-dir /tmp/*.whl && \
    useradd --create-home appuser

USER appuser
EXPOSE 8000

CMD ["uvicorn", "warehouse_manager.api:app", "--host", "0.0.0.0", "--port",
"8000"]
```

Результати:

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab10.py> .venv\Scripts\activate
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab10.py> python -m venv .venv
Unable to copy 'C:\\Program Files\\WindowsApps\\PythonSoftwareFoundation.Python.3.13_3
сія\\PycharmProjects\\lab10.py\\.venv\\Scripts\\python.exe'
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab10.py> .venv\Scripts\activate
```

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab10.py> pip install -e ".[dev]"
Obtaining file:///C:/Users/%D0%90%D0%BD%D0%B0%D1%81%D1%82%D0%B0%D1%81%D1%96%D1%8F/PycharmProjects/lab10.py
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Collecting fastapi>=0.110 (from warehouse-manager==1.0.0)
  Using cached fastapi-0.159.1-py3-none-any.whl.metadata (26 kB)
Collecting uvicorn>=0.30 (from warehouse-manager==1.0.0)
  Using cached uvicorn-0.54.0-py3-none-any.whl.metadata (6.6 kB)
Collecting sqlalchemy>=2.0 (from warehouse-manager==1.0.0)
  Using cached sqlalchemy-2.1.6-cp313-cp313-win_amd64.whl.metadata (9.9 kB)
Collecting pydantic>=2.0 (from warehouse-manager==1.0.0)
  Using cached pydantic-2.14.1-py3-none-any.whl.metadata (133 kB)
Collecting pydantic-settings>=2.0 (from warehouse-manager==1.0.0)
  Downloading pydantic_settings-2.16.1-py3-none-any.whl.metadata (4.0 kB)
Collecting httpx>=0.27 (from warehouse-manager==1.0.0)
  Using cached httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting pytest>=8.0 (from warehouse-manager==1.0.0)
```

```
 Building wheels for collected packages: warehouse-manager
   Building editable for warehouse-manager (pyproject.toml) ... done
   Created wheel for warehouse-manager: filename=warehouse_manager-1.0.0-0.editable-py3-none-any.whl size=3475 sha256=d
6
   Stored in directory: C:\Users\Анастасія\AppData\Local\Temp\pip-ephem-wheel-cache-7id_3h09\wheels\29\1b\63\ed8f576fe5
 Successfully built warehouse-manager
 Installing collected packages: typing-extensions, ruff, python-dotenv, pyproject_hooks, pygments, pluggy, pathspec, pa
 a, click, certifi, ast-serialize, annotated-types, annotated-doc, uvicorn, typing-inspection, sqlalchemy, pytest, pyda
 arlette, pytest-asyncio, pydantic, httpx, pydantic-settings, fastapi, warehouse-manager
 Successfully installed annotated-doc-0.0.5 annotated-types-0.8.0 anyio-4.15.1 ast-serialize-0.13.1 build-1.6.1 certifi
 6.0 httpcore-1.0.9 httpx-0.28.1 idna-3.20 iniconfig-2.3.1 librt-0.16.0 mypy-2.4.0 mypy_extensions-1.1.0 opentelemetry-
 -2.14.1 pydantic-core-2.50.1 pydantic-settings-2.16.1 pygments-2.21.0 pyproject_hooks-1.3.3 pytest-9.1.1 pytest-asynci
 ette-1.9.0 typing-extensions-4.16.0 typing-inspection-0.4.4 uvicorn-0.54.0 warehouse-manager-1.0.0

 [notice] A new release of pip is available: 26.1.2 -> 26.2.2
 [notice] To update, run: python.exe -m pip install --upgrade pip
```

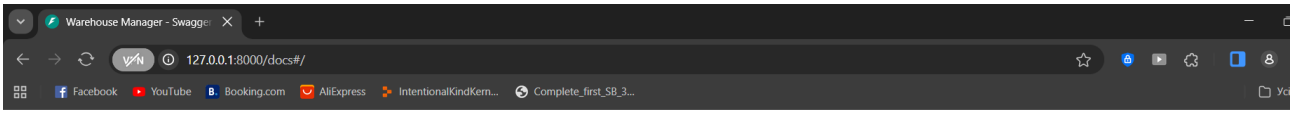
```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab10.py> pytest -v
============================= test session starts ==============================
platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\Анастасія\PycharmProjects\lab10.py\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Анастасія\PycharmProjects\lab10.py
configfile: pyproject.toml
plugins: anyio-4.15.1, asyncio-1.4.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 6 items

tests/test_api.py::test_health PASSED                                    [ 16%]
tests/test_repositories.py::test_create_and_get_product PASSED           [ 33%]
tests/test_repositories.py::test_add_incoming_and_write_off PASSED       [ 50%]
tests/test_services.py::test_service_register_incoming PASSED            [ 66%]
tests/test_services.py::test_service_invalid_incoming_amount PASSED      [ 83%]
tests/test_services.py::test_service_excessive_write_off PASSED          [100%]

============================== warnings summary ===============================
```

```
========================== 6 passed, 4 warnings in 0.07s ===========================
```

```
(.venv) PS C:\Users\Анастасія\PycharmProjects\lab10.py> uvicorn warehouse_manager.api:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\Анастасія\\PycharmProjects\\lab10.py']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [17436] using StatReload
2026-10-22 03:37:38,230 [INFO] warehouse_manager: Warehouse Manager started
INFO:     Started server process [22836]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```



Warehouse Manager 0.1.0 OAS 3.1

/openapi.json

default

- GET /health Health
- GET /products Get Products
- POST /products Create Product
- POST /products/{id}/incoming Product Incoming
- POST /products/{id}/write-off Product Write Off
- GET /stock/operations Get Stock Operations

Schemas

- HTTPValidationError > Expand all object
- OperationRequest > Expand all object
- ProductCreate > Expand all object
- ValidationError > Expand all object

POST /products Create Product

Parameters | Cancel | Reset

No parameters

Request body required — application/json

Edit Value | Schema

```
{
  "code": "PRD-001",
  "name": "Ноутбук Lenovo",
  "quantity": 15,
  "price": 25999.99
}
```

Server response

Code	Details
201	Response body

```
{
  "id": 2,
  "code": "PRD-002",
  "name": "Ноутбук Lenovo",
  "quantity": 15,
  "price": 25999.99
}
```

Download

POST /products/{id}/incoming Product Incoming

Parameters | Cancel | Reset

Name	Description
id * required integer (path)	2

Request body required — application/json

Edit Value | Schema

```
{
  "amount": 3
}
```

Code Details

200

Response body


```
{
  "id": 2,
  "code": "PRD-002",
  "name": "Ноутбук Lenovo",
  "quantity": 21,
  "price": 25999.99
}
```


Download

Response headers

POST /products/{id}/write-off Product Write Off

Parameters Cancel Reset

Name Description

id * required
integer
(path)

2

Request body required

application/json

Edit Value | Schema


```
{
  "amount": 10
}
```


Code Details

200

Response body


```
{
  "id": 2,
  "code": "PRD-002",
  "name": "Ноутбук Lenovo",
  "quantity": 11,
  "price": 25999.99
}
```


Download

Code Details

200

Response body


```
[
  {
    "id": 1,
    "product_id": 2,
    "operation_type": "incoming",
    "amount": 3,
    "timestamp": "2026-10-22T00:57:48.585236"
  },
  {
    "id": 2,
    "product_id": 2,
    "operation_type": "incoming",
    "amount": 3,
    "timestamp": "2026-10-22T00:59:43.420058"
  },
  {
    "id": 3,
    "product_id": 2,
    "operation_type": "write-off",
    "amount": 10,
    "timestamp": "2026-10-22T01:00:14.785126"
  }
]
```


Download

GET /products Get Products

Parameters Cancel

No parameters

Execute Clear

Code Details

200

Response body


```
[
  {
    "id": 1,
    "code": "PRD-001",
    "name": "Ноутбук Lenovo",
    "quantity": 15,
    "price": 25999.99
  },
  {
    "id": 2,
    "code": "PRD-002",
    "name": "Ноутбук Lenovo",
    "quantity": 11,
    "price": 25999.99
  }
]
```

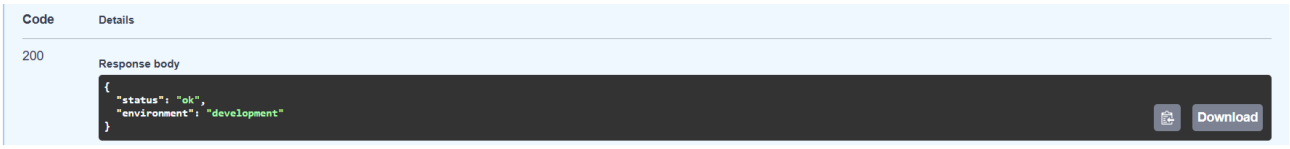

Download

GET /health Health

Parameters Cancel

No parameters

Execute Clear



Висновок: у ході виконання лабораторної роботи було успішно створено та налаштовано повноцінний вебзастосунок «Система обліку складських запасів» (warehouse-manager) із дотриманням професійного стандарту архітектури src-layout. У межах проєкту реалізовано моделі даних на базі SQLAlchemy, шари репозиторіїв і сервісів для бізнес-логіки, а також надійний API-інтерфейс на FastAPI. Окрім цього, було успішно налаштовано схеми валідації та серіалізації через Pydantic, опрацьовано обмеження цілісності бази даних і забезпечено стабільну роботу всіх маршрутів для управління товарами та складськими операціями.